

# 18-VITALA-ARCHITECTURE-V1.md

## Partie 3 — Architecture technique et gouvernance

---

### 16. Objectif

L'architecture technique de Vitala définit les composants techniques, leurs responsabilités et les règles de développement qui garantissent la stabilité, la performance et l'évolutivité de la plateforme.

Elle constitue le référentiel officiel pour tous les développements réalisés sur le projet.

---

### 17. Architecture cible

L'architecture cible de Vitala repose sur un **monorepo modulaire** permettant de partager les modèles métier, les moteurs et les services entre plusieurs clients.

Les composants principaux sont :

- PWA React (client de référence)
- Future application Flutter
- Backend Supabase
- Base de données PostgreSQL
- Stockage des médias
- Services temps réel
- API
- Workflow Engine
- Moteurs métier

Cette architecture permet à plusieurs interfaces d'utiliser le même cœur fonctionnel.

---

### 18. Le Workflow Engine

Le Workflow Engine est l'orchestrateur de Vitala.

Il ne contient aucune logique métier propre.

Son rôle est de coordonner les traitements impliquant plusieurs moteurs.

Par exemple :

- publication d'un Flash ;
- validation d'une mission ;

- évolution du Trust ;
- création d'une notification ;
- déclenchement d'une recommandation ;
- démarrage ou arrêt d'une Veille.

Le Workflow Engine garantit que chaque moteur reste indépendant tout en participant à des traitements complexes.

---

## 19. Monorepo

Le projet est organisé sous forme de monorepo.

Cette organisation permet :

- le partage des modèles ;
- le partage des types ;
- le partage des règles métier ;
- le partage des moteurs ;
- le partage des composants communs.

Chaque application utilise uniquement les modules dont elle a besoin.

---

## 20. Préparation au multi-client

La PWA React constitue le client officiel de référence.

L'architecture est cependant conçue pour permettre l'arrivée future :

- d'une application Flutter ;
- d'interfaces partenaires ;
- d'API publiques ;
- de nouveaux clients spécialisés.

Les moteurs et les domaines métier restent indépendants des interfaces.

---

## 21. Gouvernance du code

Tout nouveau développement doit respecter les règles suivantes :

- une responsabilité par module ;
- aucune dépendance circulaire ;
- aucune duplication de logique métier ;
- séparation stricte entre interface, métier et persistance ;
- documentation obligatoire pour toute évolution majeure ;
- compatibilité avec les principes de l'architecture officielle.

---

## 22. Organisation des modules

Chaque module comprend, lorsque cela est nécessaire :

- modèles ;
- services ;
- composants ;
- hooks ;
- moteurs ;
- validations ;
- tests ;
- documentation.

L'objectif est que chaque domaine puisse être compris et maintenu indépendamment.

---

## 23. Gestion des dépendances

Les dépendances entre modules sont limitées au strict nécessaire.

Les échanges s'effectuent au travers :

- des modèles métier ;
- des événements ;
- des interfaces publiques ;
- des services transversaux.

Les appels directs entre moteurs sont interdits.

---

## 24. Observabilité

L'architecture prévoit des mécanismes permettant de suivre le fonctionnement de la plateforme :

- journalisation des événements ;
- suivi des synchronisations ;
- traçabilité des workflows ;
- surveillance des performances ;
- détection des erreurs.

Ces informations facilitent la maintenance et les audits.

---

## 25. Sécurité

La sécurité est intégrée à tous les niveaux :

- authentification ;
- autorisations ;
- politiques RLS ;
- validation des données ;
- chiffrement des informations sensibles ;
- protection des API ;
- contrôle des accès.

Aucune fonctionnalité ne peut contourner ces mécanismes.

---

## 26. Évolutivité

L'architecture est conçue pour évoluer progressivement.

Les futurs développements pourront notamment intégrer :

- Intelligence artificielle avancée ;
- organisations et entreprises ;
- marketplace de services ;
- extension multi-pays ;
- nouveaux moteurs spécialisés ;
- nouveaux clients.

Ces évolutions devront respecter les principes définis dans ce document.

---

## 27. Cycle de développement

Toute évolution suit les étapes suivantes :

1. Définition du besoin.
2. Mise à jour de la documentation.
3. Validation de l'architecture.
4. Développement.
5. Tests.
6. Revue.
7. Déploiement.
8. Suivi en production.

La documentation officielle précède toujours les développements majeurs.

---

## 28. Constitution technique

Le présent document constitue la Constitution technique de Vitala.

Toute décision architecturale importante doit être compatible avec les principes définis ici.

En cas de conflit entre une implémentation et cette architecture, l'architecture fait référence jusqu'à sa révision officielle.

---

## 29. Conclusion

L'architecture de Vitala repose sur une vision modulaire, orientée domaines, orientée moteurs et pensée pour durer.

Elle privilégie l'indépendance des composants, la documentation vivante, la sécurité, la performance et l'évolutivité.

Grâce à cette organisation, la plateforme pourra accueillir de nouveaux moteurs, de nouveaux clients, de nouveaux domaines et de nouvelles technologies sans remettre en cause ses fondations.

---

---

### Statut

 Verrouillé

### Document

18-VITALA-ARCHITECTURE-V1.md

### Version

1.0

### Documents liés

- 01-VITALA-VISION-V1.md
- 03-VITALA-TRUST-LEVELS-V1.md
- 03A-VITALA-TRUST-ENGINE-V1.md
- 14-VITALA-DOMAIN-MODEL-V1.md
- 15-VITALA-DATABASE-V1.md
- 16-VITALA-API-V1.md
- 17-VITALA-DESIGN-SYSTEM-V1.md
- 19-VITALA-MVP-V1.md
- 20-VITALA-DEPLOYMENT-V1.md

---